Online Shoppers' Purchasing Intention Prediction Using Machine Learning

Project Report

Prepared from the uploaded online_shoppers_intention.csv dataset and final_code_updated.py.

Abstract

This project uses online shoppers' browsing-session information to predict whether a session results in a purchase. The target column is Revenue: True means a purchase and False means no purchase. The uploaded dataset contains 12,330 rows and 18 columns. The Python implementation prepares features, compares classifiers using ROC-AUC cross-validation, trains a classifier, and evaluates predictions. The intended application is to support e-commerce analysis and website conversion improvements.

1. Introduction

E-commerce websites collect session-level data that describe how visitors interact with a website. Predicting purchase intention from these signals is a binary classification task. A prediction estimates the likelihood of a purchase; it does not guarantee a customer's action.

2. Problem Statement

Build a supervised machine-learning classifier that uses website session attributes to predict the Revenue label: purchase or no purchase.

3. Objectives

- Explore the dataset and the distribution of the Revenue target.
- Prepare numeric and categorical features for classification.
- Address class imbalance with SMOTE inside the training pipeline.
- Compare several classifiers using cross-validation ROC-AUC.
- Evaluate the selected model using accuracy, F1-score, ROC-AUC and a classification report.
- Identify limitations and improvements to make evaluation more reliable.

4. Dataset Description

The uploaded CSV has 12,330 records and 18 columns: 17 candidate input features and the Revenue target. The target distribution is 10,422 False labels and 1,908 True labels. Therefore, approximately 15.47% of sessions have Revenue=True and 84.53% have Revenue=False. This imbalance means accuracy alone is not enough to evaluate the model.

- Page activity: Administrative, Informational, ProductRelated and corresponding duration features.
- Session behavior: BounceRates, ExitRates, PageValues and SpecialDay.
- Context: Month, OperatingSystems, Browser, Region, TrafficType, VisitorType and Weekend.
- Target: Revenue (purchase / no purchase).

5. Methodology

5.1 Data loading

Read the CSV using pandas and inspect its shape, columns, data types and target distribution.

5.2 Feature preparation

The original code converts Weekend and Revenue to 0/1, creates a Returning_Visitor indicator, drops VisitorType and ordinal-encodes Month. These transformations should be reviewed: Month is categorical, and the Returning_Visitor flag loses distinctions between New_Visitor and other categories.

5.3 Train-test split

The original code uses a 70/30 split with random_state=0 but does not stratify by Revenue. Stratification is recommended to preserve the target ratio in both subsets.

5.4 Preprocessing

The original pipeline scales numeric features using MinMaxScaler and one-hot encodes object columns. Imputation is also included for numeric data.

5.5 Class imbalance

SMOTE creates synthetic examples for the minority class. It must be applied only to each training fold within a pipeline, never to the full dataset before splitting.

5.6 Feature selection

The original code uses SelectKBest with chi-squared scoring and k=6. That fixed number should be justified through validation; testing k='all' or tuning k is a useful comparison.

5.7 Model comparison

The supplied code compares Random Forest, Decision Tree, KNN, Ridge Classifier, Bernoulli Naive Bayes and SVC using 10-fold ROC-AUC cross-validation.

5.8 Final evaluation

The original script always trains SVC, regardless of the cross-validation ranking. A better workflow selects the highest-ranked model using training data only, then evaluates it on a held-out test set.

6. Algorithms Used

- Random Forest: combines decision trees to model nonlinear patterns.
- Decision Tree: learns rule-based splits from feature values.

- K-Nearest Neighbors: predicts using nearby training samples.
- Ridge Classifier: linear classifier with L2 regularization.
- Bernoulli Naive Bayes: probabilistic classifier based on Bernoulli-style features.
- Support Vector Classifier: learns a decision boundary and can model nonlinear patterns with its default kernel.

7. Evaluation Metrics

- Accuracy: fraction of predictions that are correct.
- Precision: fraction of predicted purchases that are actual purchases.
- Recall: fraction of actual purchases found by the model.
- F1-score: harmonic mean of precision and recall.
- ROC-AUC: measures ranking quality across thresholds; calculate it from probabilities or decision scores, not only hard class predictions.
- Confusion matrix: summarizes true negatives, false positives, false negatives and true positives.

8. Results and Current Status

Inspection of the uploaded CSV confirms 12,330 rows, 18 columns, 10,422 no-purchase sessions and 1,908 purchase sessions. The original script first failed in this review because it could not find the CSV when launched from a different working directory. Running it from the dataset directory entered the model-comparison stage but exceeded the available execution time. Consequently, no trustworthy test scores or winning model can be reported from this run. Run the corrected script to generate actual results.

9. Issues Identified in the Original Code

- The relative CSV path can cause FileNotFoundError when the current working directory differs.
- The train-test split is not stratified despite class imbalance.
- Ordinal encoding Month imposes an arbitrary order on month names.
- Reducing VisitorType to a Returning_Visitor flag loses category information.
- SVC is hard-coded as the final model even if another model ranks higher.
- ROC-AUC is computed from predicted class labels rather than continuous scores.
- The variable named f1_score shadows the imported metric function.
- Ten-fold cross-validation across six SMOTE pipelines can be slow.
- Suppressing all warnings can hide useful diagnostics.

10. Recommended Improvements

- Use a path relative to the script and show a clear error if the CSV is missing.
- Use a stratified train-test split and a fixed random seed.
- Keep preprocessing and SMOTE inside the cross-validation pipeline.
- One-hot encode nominal categories such as Month and VisitorType.
- Select the final classifier using cross-validation rather than hard-coding SVC.

- Calculate ROC-AUC using predict_proba or decision_function.
- Report purchase-class precision, recall and F1 alongside accuracy.
- Compare SMOTE against class-weighted models and tune hyperparameters on training data only.

11. Applications

- Analyzing browsing patterns associated with purchase conversion.
- Helping e-commerce teams investigate page-value and exit-rate behavior.
- Supporting aggregate conversion analysis and website experiments.
- Prioritizing further analysis of sessions that may need improved user experience.

12. Limitations

The model uses historical session data and may not generalize to every store or time period. Correlation does not establish causation. SMOTE-generated records may not perfectly represent real visitors. Performance, calibration and business impact should be validated before deployment. This report does not invent metrics because the original code did not complete successfully in the review environment.

13. Conclusion

This project formulates online shopping purchase intention as a binary classification problem. The uploaded implementation includes preprocessing, SMOTE, feature selection, model comparison and evaluation. Important improvements are reliable file loading, stratified splitting, appropriate categorical encoding, choosing the best model from validation results, and computing ROC-AUC from continuous scores. Insert actual test metrics after the revised script finishes.

References

- Sakar, C. O., et al. (2018). Real-time prediction of online shoppers' purchasing intention using multilayer perceptron and LSTM recurrent neural networks. Neural Computing and Applications.
- scikit-learn documentation: preprocessing, pipelines, model selection and classification metrics.
- imbalanced-learn documentation: SMOTE and pipeline usage.